is allowed. That way, if a CORS request is made to a server that doesn't support CORS or doesn't have the right CORS headers, the request fails.

1.3.3 Flexibility

CORS gives servers the flexibility to configure cross-origin access in a variety of ways. The server can specify various features:

- Which domains are allowed to make requests
- Which HTTP methods are allowed (for example, `GET/PUT/POST/DELETE`)
- Which headers are allowed on the HTTP request
- Whether or not requests may include cookie data
- Which response headers the client can read

These rich settings put the server firmly in control of how the CORS request works.

1.3.4 Easy for developers

Because CORS is a standardized specification, it works in a consistent manner across sites. Developers need only learn about CORS once; then they can use the same techniques across all sites that support CORS.

While CORS requires new configuration on the server side, the client-side developer experience remains largely unchanged. JavaScript's `XMLHttpRequest` object has been available in browsers for over 10 years. Developers make CORS requests using the same `XMLHttpRequest` object they are familiar with. There isn't any new code for the developer to learn. From the developer's perspective, same-origin and cross-origin requests look mostly the same. (There are some slight differences, which we'll cover in chapter 2.)

1.3.5 Reduced maintenance overhead

There are ways to make cross-origin requests without using CORS (appendix D covers some of these techniques). But these techniques require custom code, custom servers, or additional documentation. This leads to an additional maintenance burden for the server developer.

Conversely, CORS only requires a few additional response headers. This reduced maintenance means that API owners can focus their attention on other things, rather than worrying about reinventing and maintaining new cross-domain mechanisms. Because CORS is a published specification with broad browser support, site owners can rest assured that their implementation is stable and that details won't change.

1.4 Summary

CORS allows client code to make cross-origin requests to remote servers. CORS is necessary because the browser's same-origin policy traditionally disallows cross-origin requests, which makes it difficult to load data from other sites. Here are some benefits of CORS: